Cognizant - UpGrad Training Program
WEEKLY CODING SUBMISSION
Name : Durgesh Vijay Bhangale
Week : Week 3
Trainer : Kunal

Day 1

Question 1 : Student Result Analyzer (Method Overloading + Collections)

```
class StudentResultAnalyzer
{
   public static double CalculateAverage(List<int> marks)
   {
      int sum = 0;
      foreach (int m in marks)
      {
         sum += m;
      }
      return sum / (double)marks.Count;
   }
   public static double CalculateAverage(int m1, int m2, int m3, int m4, int m5)
   {
      return (m1 + m2 + m3 + m4 + m5) / 5.0;
   }
   public static void PrintGrade(double average)
   {
      if (average > 80)
         Console.WriteLine($"Average: {average:F2} → Grade A");
      else if (average > 60)
         Console.WriteLine($"Average: {average:F2} → Grade B");
      else if (average > 40)
         Console.WriteLine($"Average: {average:F2} → Grade C");
      else
         Console.WriteLine($"Average: {average:F2} → Fail");
   }

   static void Main()
   {       List<int> marksList = new List<int> { 85, 78, 92, 88, 76 };
      double avg1 = CalculateAverage(marksList);
      PrintGrade(avg1);
      double avg2 = CalculateAverage(45, 55, 60, 50, 40);
      PrintGrade(avg2);
   }
}
```

Question 2 : Custom Calculator with Operation Method

```
using System;

class CustomCalculator
{
    public static double Calculate(double a, double b, string operation)
    {
        return operation.ToLower() switch
        {
            "add"      => a + b,
            "subtract" => a - b,
            "multiply" => a * b,
            "divide"   => a
        };
    }

    static void Main()
    {
        Console.WriteLine("Add: " + Calculate(10, 5, "add"));
        Console.WriteLine("Subtract: " + Calculate(10, 5, "subtract"));
        Console.WriteLine("Multiply: " + Calculate(10, 5, "multiply"));
        Console.WriteLine("Divide: " + Calculate(10, 0, "divide"));
    }
}
```

Question 3 : Recursive Method – Power Calculator

```
using System;

class PowerCalculator
{
    public static int Power(int baseNum, int exponent)
    {
        if (exponent == 0)
            return 1;
        return baseNum * Power(baseNum, exponent - 1);
    }

    static void Main()
    {
        Console.WriteLine("2^3 = " + Power(2, 3));
        Console.WriteLine("5^4 = " + Power(5, 4));
        Console.WriteLine("7^0 = " + Power(7, 0));
    }
}
```

Question 4 : Dictionary-Based Product Inventory

```
using System;
using System.Collections.Generic;

class ProductInventory
{
    private Dictionary<string, double> products = new Dictionary<string, double>();
    public void AddProduct(string name, double price)
    {
        products[name] = price;
    }
    public bool RemoveProduct(string name)
    {
        return products.Remove(name);
    }
    public void DisplayAllProducts()
    {
        Console.WriteLine("Product Inventory:");
        foreach (var item in products)
        {
            Console.WriteLine($"{item.Key} → ₹{item.Value}");
        }
    }
    public double FindProductPrice(string name)
    {
        return products.ContainsKey(name) ? products[name] : -1;
    }

    static void Main()
    {
        ProductInventory inventory = new ProductInventory();
        inventory.AddProduct("Laptop", 55000);
        inventory.AddProduct("Phone", 25000);
        inventory.AddProduct("Headphones", 3000);

        inventory.DisplayAllProducts();
        Console.WriteLine("Price of Phone: ₹" + inventory.FindProductPrice("Phone"));
        inventory.RemoveProduct("Headphones");
        inventory.DisplayAllProducts();
    }
}
```

Question 5 : Build a Student Management Class

```
using System;
using System.Collections.Generic;

class Student
{
   public int Id { get; set; }
   public string Name { get; set; }
   public int Marks { get; set; }

   public string CalculateGrade()
   {
      if (Marks >= 85) return "A";
      else if (Marks >= 70) return "B";
      else if (Marks >= 50) return "C";
      else return "Fail";
   }
}

class Program
{
   static void Main()
   {
      List<Student> students = new List<Student>
      {
         new Student { Id = 1, Name = "Durgesh", Marks = 90 },
         new Student { Id = 2, Name = "Yash", Marks = 72 },
         new Student { Id = 3, Name = "Akash", Marks = 55 },
         new Student { Id = 4, Name = "Suyash", Marks = 35 }
      };

      foreach (var s in students)
      {
         Console.WriteLine($"ID: {s.Id}, Name: {s.Name}, Marks: {s.Marks}, Grade:
{s.CalculateGrade()}");
      }
   }
}
```

Question 6 : **Product Class with Constructor Overloading**

```
using System;

class Product
{
   public string Name { get; set; }
   public double Price { get; set; }
   public int Quantity { get; set; }
```

```
    public Product()
    {
        Name = "Unknown";
        Price = 0;
        Quantity = 0;
    }
    public Product(string name, double price, int quantity)
    {
        Name = name;
        Price = price;
        Quantity = quantity;
    }
    public double GetTotalPrice()
    {
        return Price * Quantity;
    }
}

class Program
{
    static void Main()
    {
        Product p1 = new Product();
        Console.WriteLine($"Product: {p1.Name}, Total Price: {p1.GetTotalPrice()}");

        Product p2 = new Product("Laptop", 55000, 2);
        Console.WriteLine($"Product: {p2.Name}, Total Price: {p2.GetTotalPrice()}");
    }
}
```

Question 7 : Encapsulation with Validation

```
using System;

class Employee
{
    private double salary;
    private int age;
    public double Salary
    {
        get { return salary; }
        set
        {
            if (value > 0)
                salary = value;
            else
                Console.WriteLine("Salary must be greater than 0.");
        }
    }
```

```
    public int Age
    {
        get { return age; }
        set
        {
            if (value >= 18 && value <= 60)
                age = value;
            else
                Console.WriteLine("Age must be between 18 and 60.");
        }
    }

    public void Display()
    {
        Console.WriteLine($"Employee Age: {age}, Salary: {salary}");
    }
}
```

Question 8 : Shape Area Calculator (Runtime Polymorphism)

```
using System;
using System.Collections.Generic;
abstract class Shape
{
    public abstract double CalculateArea();
}

class Circle : Shape
{
    public double Radius { get; set; }
    public Circle(double radius) { Radius = radius; }

    public override double CalculateArea()
    {
        return Math.PI * Radius * Radius;
    }
}

class Rectangle : Shape
{
    public double Width { get; set; }
    public double Height { get; set; }
    public Rectangle(double width, double height) { Width = width; Height = height; }

    public override double CalculateArea()
    {
        return Width * Height;
    }
}
```

```
class Triangle : Shape
{
   public double Base { get; set; }
   public double Height { get; set; }
   public Triangle(double b, double h) { Base = b; Height = h; }

   public override double CalculateArea()
   {
      return 0.5 * Base * Height;
   }
}

class Program
{
   static void Main()
   {
      List<Shape> shapes = new List<Shape>
      {
         new Circle(5),
         new Rectangle(4, 6),
         new Triangle(3, 7)
      };
      foreach (var shape in shapes)
      {
         Console.WriteLine($"Area: {shape.CalculateArea():F2}");
      }
   }
}
```

Question 9 : Bank Account System (Method Overriding)

```
using System;
using System.Collections.Generic;
class Account
{
   public double Balance { get; set; }

   public Account(double balance)
   {
      Balance = balance;
   }
   public virtual double CalculateInterest()
   {
      return Balance * 0.01;
   }
}
class SavingsAccount : Account
{
   public SavingsAccount(double balance) : base(balance) { }
```

```
    public override double CalculateInterest()
    {
        return Balance * 0.04;
    }
}
class CurrentAccount : Account
{
    public CurrentAccount(double balance) : base(balance) { }

    public override double CalculateInterest()
    {
        return Balance * 0.02;
    }
}

class Program
{
    static void Main()
    {
        List<Account> accounts = new List<Account>
        {
            new SavingsAccount(10000),
            new CurrentAccount(20000),
            new Account(5000)
        };

        foreach (var acc in accounts)
        {
            Console.WriteLine($"{acc.GetType().Name} → Interest: {acc.CalculateInterest()}");
        }
    }
}
```

Question 10 : Vehicle Service System (Polymorphism + Encapsulation)

```
using System;
using System.Collections.Generic;
class Vehicle
{
    private string name;

    public Vehicle(string name)
    {
        this.name = name;
    }

    public string Name
```

```
    {
        get { return name; }
        set { name = value; }
    }

    public void StartEngine()
    {
        Console.WriteLine($"{Name} engine started.");
    }
    public virtual double ServiceCost()
    {
        return 0;
    }
}
class Car : Vehicle
{
    public Car(string name) : base(name) { }

    public override double ServiceCost()
    {
        return 5000;
    }
}
class Bike : Vehicle
{
    public Bike(string name) : base(name) { }

    public override double ServiceCost()
    {
        return 2000;
    }
}
class Truck : Vehicle
{
    public Truck(string name) : base(name) { }

    public override double ServiceCost()
    {
        return 10000;
    }
}

class Program
{
    static void Main()
    {
        List<Vehicle> vehicles = new List<Vehicle>
        {
            new Car("Honda City"),
            new Bike("Yamaha R15"),
```

```
        new Truck("Volvo Truck")
    };

    double totalCost = 0;

    foreach (var v in vehicles)
    {
        v.StartEngine();
        double cost = v.ServiceCost();
        Console.WriteLine($"{v.GetType().Name} ({v.Name}) → Service Cost: {cost}");
        totalCost += cost;
    }

    Console.WriteLine($"Total Service Cost: {totalCost}");
  }
}
```

Day 2, 3

Question 1 : Even-Odd Transformation Pipeline

Use:

- **LINQ only**
- **No manual loops**

```
using System;
using System.Linq;

class Program
{
   static void Main()
   {
      int[] numbers = { 3, 6, 9, 12, 3, 6, 15, 18, 21 };

      var result = numbers
         .Distinct()
         .Where(n => n % 3 == 0)
         .Select(n => n * n)
         .OrderBy(n => n)
         .ToList();

      Console.WriteLine("Transformed Result:");
      result.ForEach(n => Console.WriteLine(n));
   }
}
```

Question 2 : **Custom Stack with History Tracking**

Must use:

- Stack
- Encapsulation
- Custom exception for empty stack

```
using System;
using System.Collections.Generic;

class EmptyStackException : Exception
{
   public EmptyStackException(string message) : base(message) { }
}

class CustomStack
{
   private Stack<int> mainStack = new Stack<int>();
```

```
private Stack<string> history = new Stack<string>();

public void Push(int value)
{
    mainStack.Push(value);
    history.Push($"PUSH {value}");
}

public int Pop()
{
    if (mainStack.Count == 0)
        throw new EmptyStackException("Stack is empty. Cannot Pop.");

    int val = mainStack.Pop();
    history.Push($"POP {val}");
    return val;
}

public int Peek()
{
    if (mainStack.Count == 0)
        throw new EmptyStackException("Stack is empty. Cannot Peek.");

    return mainStack.Peek();
}

public void UndoLastOperation()
{
    if (history.Count == 0)
    {
        Console.WriteLine("No operations to undo.");
        return;
    }

    string lastOp = history.Pop();

    if (lastOp.StartsWith("PUSH"))
    {
        if (mainStack.Count > 0)
            mainStack.Pop();
        Console.WriteLine($"Undo: {lastOp}");
    }
    else if (lastOp.StartsWith("POP"))
    {
        string[] parts = lastOp.Split(' ');
        int val = int.Parse(parts[1]);
        mainStack.Push(val);
        Console.WriteLine($"Undo: {lastOp}");
    }
}
```

```
    public void Display()
    {
        Console.WriteLine("Stack contents: " + string.Join(", ", mainStack));
    }
}

class Program
{
    static void Main()
    {
        CustomStack stack = new CustomStack();

        stack.Push(10);
        stack.Push(20);
        stack.Display();

        stack.Pop();
        stack.Display();

        stack.UndoLastOperation();
        stack.Display();

        stack.UndoLastOperation();
        stack.Display();
    }
}
```

Question 3 : **Task Scheduler with Queue + Priority**

```
using System;
using System.Collections.Generic;

class TaskScheduler
{
    private Queue<string> high = new Queue<string>();
    private Queue<string> medium = new Queue<string>();
    private Queue<string> low = new Queue<string>();

    public void AddTask(string task, int priority)
    {
        if (priority == 1) high.Enqueue(task);
        else if (priority == 2) medium.Enqueue(task);
        else low.Enqueue(task);
    }

    public void ProcessTasks()
    {
        while (high.Count > 0) Console.WriteLine("High → " + high.Dequeue());
```

```
        while (medium.Count > 0) Console.WriteLine("Medium → " + medium.Dequeue());
        while (low.Count > 0) Console.WriteLine("Low → " + low.Dequeue());
    }
}

class Program
{
    static void Main()
    {
        TaskScheduler scheduler = new TaskScheduler();

        scheduler.AddTask("Pay bills", 2);
        scheduler.AddTask("Fix server", 1);
        scheduler.AddTask("Buy groceries", 3);
        scheduler.AddTask("Prepare report", 1);

        scheduler.ProcessTasks();
    }
}
```

Question 4 : Improved HashMap with Collision Detection

```
using System;
using System.Collections.Generic;

class SimpleHashMap
{
    private List<(string key, string val)>[] buckets;
    public SimpleHashMap(int size) => buckets = new List<(string, string)>[size];

    private int Index(string key) => Math.Abs(key.GetHashCode()) % buckets.Length;

    public void Add(string key, string val)
    {
        int i = Index(key);
        if (buckets[i] == null) buckets[i] = new List<(string, string)>();
        if (buckets[i].Count > 0) Console.WriteLine($"⚠ Collision at {i}");
        buckets[i].Add((key, val));
    }

    public string Get(string key)
    {
        int i = Index(key);
        return buckets[i]?.Find(x => x.key == key).val;
    }

    public void Remove(string key)
    {
```

```
        int i = Index(key);
        buckets[i]?.RemoveAll(x => x.key == key);
    }

    public void Display()
    {
        for (int i = 0; i < buckets.Length; i++)
            if (buckets[i] != null)
                Console.WriteLine($"Bucket {i}: " + string.Join(", ", buckets[i]));
    }
}

class Program
{
    static void Main()
    {
        var map = new SimpleHashMap(3);
        map.Add("apple", "red");
        map.Add("banana", "yellow");
        map.Add("grape", "purple");
        Console.WriteLine("Get banana → " + map.Get("banana"));
        map.Remove("apple");
        map.Display();
    }
}
```

Question 5 : Binary Tree – Count & Height

```
using System;

class Node
{
    public int Data;
    public Node Left, Right;
    public Node(int data) { Data = data; }
}

class BinaryTree
{
    public Node Root;

    public int CountNodes(Node node)
    {
        if (node == null) return 0;
        return 1 + CountNodes(node.Left) + CountNodes(node.Right);
    }

    public int GetHeight(Node node)
```

```
    {
        if (node == null) return 0;
        return 1 + Math.Max(GetHeight(node.Left), GetHeight(node.Right));
    }

    public int FindMin(Node node)
    {
        if (node.Left == null) return node.Data;
        return FindMin(node.Left);
    }


    public int FindMax(Node node)
    {
        if (node.Right == null) return node.Data;
        return FindMax(node.Right);
    }
}

class Program
{
    static void Main()
    {
        BinaryTree tree = new BinaryTree();
        tree.Root = new Node(10);
        tree.Root.Left = new Node(5);
        tree.Root.Right = new Node(20);
        tree.Root.Left.Left = new Node(2);
        tree.Root.Left.Right = new Node(7);

        Console.WriteLine("Count: " + tree.CountNodes(tree.Root));
        Console.WriteLine("Height: " + tree.GetHeight(tree.Root));
        Console.WriteLine("Min: " + tree.FindMin(tree.Root));
        Console.WriteLine("Max: " + tree.FindMax(tree.Root));
    }
}
```

Question 6 : Student File Processor with Error Logging

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;

class InvalidMarksException : Exception
{
    public InvalidMarksException(string msg) : base(msg) { }
}
```

```
class Student
{
    public string Name { get; set; }
    public int Marks { get; set; }
}

class Program
{
    static void Main()
    {
        string inputFile = "students.txt";
        string errorFile = "errors.txt";
        List<Student> students = new List<Student>();

        foreach (var line in File.ReadAllLines(inputFile))
        {
            try
            {
                var parts = line.Split(',');
                string name = parts[0];
                if (!int.TryParse(parts[1], out int marks))
                    throw new InvalidMarksException($"Invalid marks for {name}: {parts[1]}");

                students.Add(new Student { Name = name, Marks = marks });
            }
            catch (InvalidMarksException ex)
            {
                File.AppendAllText(errorFile, ex.Message + Environment.NewLine);
            }
        }

        Console.WriteLine("Valid Students:");
        students.ForEach(s => Console.WriteLine($"{s.Name} → {s.Marks}"));

        if (students.Count > 0)
            Console.WriteLine("Average Marks: " + students.Average(s => s.Marks));
    }
}
```

Question 7 : Nullable Order Discount System

```
using System;

class Order
{
   public double Price { get; set; }
   public double? DiscountPercentage { get; set; }

   public double CalculateFinalPrice()
   {
      double discount = DiscountPercentage ?? 10;

      if (discount > 40)
         throw new Exception("Discount cannot exceed 40%");

      double finalPrice = Price - (Price * discount / 100);

      return Price switch
      {
         > 5000 => finalPrice - (Price * 0.05),
         _ => finalPrice
      };
   }
}

class Program
{
   static void Main()
   {
      try
      {
         Order order1 = new Order { Price = 6000, DiscountPercentage = null };
         Console.WriteLine("Final Price: " + order1.CalculateFinalPrice());

         Order order2 = new Order { Price = 4000, DiscountPercentage = 15 };
         Console.WriteLine("Final Price: " + order2.CalculateFinalPrice());

         Order order3 = new Order { Price = 3000, DiscountPercentage = 50 };
         Console.WriteLine("Final Price: " + order3.CalculateFinalPrice());
      }
      catch (Exception ex)
      {
         Console.WriteLine("Error: " + ex.Message);
      }
   }
}
```

Question 8 : Employee Analytics using LINQ

```
using System;
using System.Collections.Generic;
using System.Linq;

class Employee
{
   public string Name { get; set; }
   public string Department { get; set; }
   public double Salary { get; set; }
}

class Program
{
   static void Main()
   {
      var employees = new List<Employee>
      {
         new Employee { Name="Durgesh", Department="HR", Salary=40000 },
         new Employee { Name="Abhishek", Department="IT", Salary=60000 },
         new Employee { Name="Devesh", Department="IT", Salary=75000 },
         new Employee { Name="Akash", Department="Finance", Salary=50000 },
         new Employee { Name="Pranav", Department="Finance", Salary=90000 }
      };

      double avgSalary = employees.Average(e => e.Salary);

      var result = employees
         .Where(e => e.Salary > avgSalary)
         .GroupBy(e => e.Department)
         .ToDictionary(
            g => g.Key,
            g => g.OrderByDescending(e => e.Salary).First()
         );

      foreach (var kv in result)
         Console.WriteLine($"{kv.Key} → {kv.Value.Name}, {kv.Value.Salary}");
   }
}
```

Question 9 : **Tuple-Based Result Evaluation System**

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;

class StudentEvaluator
```

```
{
    public static (string grade, bool isPass) Evaluate(int marks)
    {
        return marks switch
        {
            >= 90 => ("A+", true),
            >= 75 => ("A", true),
            >= 60 => ("B", true),
            >= 40 => ("C", true),
            _ => ("Fail", false)
        };
    }
}

class Program
{
    static void Main()
    {
        string file = "results.txt";
        var students = new List<(string name, int marks)>
        {
            ("Kunal", 85),
            ("Rahul", 30),
            ("Amit", 95),
            ("Sneha", 60)
        };

        using (var writer = new StreamWriter(file))
        {
            foreach (var (name, marks) in students)
            {
                var (grade, isPass) = StudentEvaluator.Evaluate(marks);
                writer.WriteLine($"{name},{marks},{grade},{isPass}");
            }
        }


        var passed = File.ReadAllLines(file)
            .Select(line => line.Split(','))
            .Where(parts => bool.Parse(parts[3]))
            .Select(parts => $"{parts[0]} → {parts[2]}");

        Console.WriteLine("Passed Students:");
        foreach (var s in passed) Console.WriteLine(s);
    }
}
```

Question 10 : Mini File-Based CRUD Student System

```
using System;
using System.IO;
using System.Linq;
class Program
{
   static string file = "students.txt";
   static void Main()
   {
      while (true)
      {
         Console.WriteLine("\n1. Add  2. View  3. Search  4. Delete  5. Exit");
         Console.Write("Choice: ");
         string choice = Console.ReadLine();

         if (choice == "1") Add();
         else if (choice == "2") View();
         else if (choice == "3") Search();
         else if (choice == "4") Delete();
         else if (choice == "5") break;
      }
   }

   static void Add()
   {
      Console.Write("Id: "); int id = int.Parse(Console.ReadLine());
      Console.Write("Name: "); string name = Console.ReadLine();
      File.AppendAllText(file, $"{id},{name}\n");
   }
   static void View()
   {
      var lines = File.ReadAllLines(file);
      foreach (var l in lines) Console.WriteLine(l);
   }
   static void Search()
   {
      Console.Write("Search Id: "); int id = int.Parse(Console.ReadLine());
      var found = File.ReadAllLines(file)
         .FirstOrDefault(l => int.Parse(l.Split(',')[0]) == id);
      Console.WriteLine(found ?? "Not found");
   }
   static void Delete()
   {
      Console.Write("Delete Id: "); int id = int.Parse(Console.ReadLine());
      var remain = File.ReadAllLines(file)
         .Where(l => int.Parse(l.Split(',')[0]) != id);
      File.WriteAllLines(file, remain);
   }
}
```

Day 4, 5

Question 1 : Thread-Safe Order Processing System

```
using System;
using System.Collections.Generic;
using System.Threading;

class Program
{
   static void Main()
   {
      OrderService service = new OrderService();

      Thread t1 = new Thread(() => service.PlaceOrder(1, "Laptop"));
      Thread t2 = new Thread(() => service.PlaceOrder(2, "Phone"));
      Thread t3 = new Thread(() => service.PlaceOrder(1, "Tablet"));

      t1.Start();
      t2.Start();
      t3.Start();

      t1.Join();
      t2.Join();
      t3.Join();
   }
}

class OrderService
{
   private readonly object _lock = new object();
   private List<int> orderIds = new List<int>();

   public void PlaceOrder(int id, string item)
   {
      lock (_lock)
      {
         if (orderIds.Contains(id))
         {
            Console.WriteLine($"Duplicate Order {id} blocked by Thread
{Thread.CurrentThread.ManagedThreadId}");
            return;
         }
         orderIds.Add(id);
         Console.WriteLine($"Order {id} ({item}) placed by Thread
{Thread.CurrentThread.ManagedThreadId}");
      }
   }
}
```

Question 2 : Parallel File Processor with Timing

```
using System;
using System.Diagnostics;
using System.IO;
using System.Linq;
using System.Threading.Tasks;

class Program
{
   static void Main()
   {
      string[] files = { "f1.txt", "f2.txt", "f3.txt", "f4.txt", "f5.txt" };


      Stopwatch sw1 = Stopwatch.StartNew();
      int seqCount = files.Sum(f => File.ReadAllText(f).Split(' ',
StringSplitOptions.RemoveEmptyEntries).Length);
      sw1.Stop();
      Console.WriteLine($"Sequential → Words: {seqCount}, Time: {sw1.ElapsedMilliseconds} ms");


      Stopwatch sw2 = Stopwatch.StartNew();
      int parCount = 0;
      object locker = new object();

      Parallel.ForEach(files, f =>
      {
         int count = File.ReadAllText(f).Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;
         lock (locker) parCount += count;
         Debug.WriteLine($"Processed {f} on Thread {Task.CurrentId}");
      });

      sw2.Stop();
      Console.WriteLine($"Parallel → Words: {parCount}, Time: {sw2.ElapsedMilliseconds} ms");
   }
}
```

Question 3 : Async Data Aggregator

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Threading.Tasks;

class Program
```

```
{
    static async Task Main()
    {
        try
        {
            var sw = Stopwatch.StartNew();

            var usersTask = FetchUsersAsync();
            var ordersTask = FetchOrdersAsync();
            var paymentsTask = FetchPaymentsAsync();

            await Task.WhenAll(usersTask, ordersTask, paymentsTask);

            var users = usersTask.Result;
            var orders = ordersTask.Result;
            var payments = paymentsTask.Result;

            sw.Stop();
            Console.WriteLine($"Fetched {users.Count} users, {orders.Count} orders, {payments.Count}
payments in {sw.ElapsedMilliseconds} ms");
        }
        catch (Exception ex)
        {
            Console.WriteLine("Error occurred: " + ex.Message);
            Console.WriteLine("Trace: " + ex.StackTrace);
        }
    }

    static async Task<List<string>> FetchUsersAsync()
    {
        await Task.Delay(500);
        return new List<string> { "Durgesh", "Abhishek", "Devesh" };
    }

    static async Task<List<string>> FetchOrdersAsync()
    {
        await Task.Delay(700);
        return new List<string> { "Order1", "Order2" };
    }

    static async Task<List<string>> FetchPaymentsAsync()
    {
        await Task.Delay(300);
        return new List<string> { "Payment1", "Payment2", "Payment3" };
    }
}
```

Question 4 : Runtime Failure Analyzer

```
using System;

class Repository
{
   public string GetData()
   {
      Console.WriteLine("Enter Repository.GetData");
      throw new Exception("Database connection failed!");
      Console.WriteLine("Exit Repository.GetData");
   }
}

class Service
{
   private Repository repo = new Repository();

   public string Process()
   {
      Console.WriteLine("Enter Service.Process");
      string data = repo.GetData();
      Console.WriteLine("Exit Service.Process");
      return data;
   }
}

class Controller
{
   private Service service = new Service();

   public void HandleRequest()
   {
      Console.WriteLine("Enter Controller.HandleRequest");
      try
      {
         var result = service.Process();
         Console.WriteLine("Result: " + result);
      }
      catch (Exception ex)
      {
         Console.WriteLine("Exception: " + ex.Message);
         Console.WriteLine("StackTrace: " + ex.StackTrace);
      }
      Console.WriteLine("Exit Controller.HandleRequest");
   }
}

class Program
```

```
{
    static void Main()
    {
        Controller controller = new Controller();
        controller.HandleRequest();
    }
}
```

Question 5 : Secure Login System (SQLite)

```
using System;
using System.Security.Cryptography;
using System.Text;
using Microsoft.Data.Sqlite;

class Program
{
    static void Main()
    {
        string connectionString = "Data Source=mydatabase.db";

        using var connection = new SqliteConnection(connectionString);
        connection.Open();
        var create = connection.CreateCommand();
        create.CommandText = @"
            CREATE TABLE IF NOT EXISTS Users (
                Id INTEGER PRIMARY KEY AUTOINCREMENT,
                Username TEXT UNIQUE,
                PasswordHash TEXT
            );";
        create.ExecuteNonQuery();
        Console.Write("Enter username: ");
        string username = Console.ReadLine()?.Trim();
        Console.Write("Enter password: ");
        string password = Console.ReadLine();

        if (string.IsNullOrWhiteSpace(username) || string.IsNullOrWhiteSpace(password))
        {
            Console.WriteLine("Invalid input.");
            return;
        }

        string hash = ComputeSha256(password);

        try
        {
            var insert = connection.CreateCommand();
            insert.CommandText = "INSERT INTO Users (Username, PasswordHash) VALUES ($u, $p)";
            insert.Parameters.AddWithValue("$u", username);
```

```
            insert.Parameters.AddWithValue("$p", hash);
            insert.ExecuteNonQuery();
            Console.WriteLine("User registered securely.");
        }
        catch (Exception ex)
        {
            Console.WriteLine("SQL Error: " + ex.Message);
        }
        Console.Write("Login username: ");
        string loginUser = Console.ReadLine()?.Trim();
        Console.Write("Login password: ");
        string loginPass = Console.ReadLine();

        string loginHash = ComputeSha256(loginPass);

        var select = connection.CreateCommand();
        select.CommandText = "SELECT COUNT(*) FROM Users WHERE Username=$u AND
PasswordHash=$p";
        select.Parameters.AddWithValue("$u", loginUser);
        select.Parameters.AddWithValue("$p", loginHash);

        long count = (long)select.ExecuteScalar();
        Console.WriteLine(count > 0 ? "Login successful!" : "Login failed.");
    }

    static string ComputeSha256(string input)
    {
        using var sha = SHA256.Create();
        byte[] bytes = sha.ComputeHash(Encoding.UTF8.GetBytes(input));
        return Convert.ToHexString(bytes);
    }
}
```

Question 6 : Transaction-Based Money Transfer (SQLite)

```
using System;
using Microsoft.Data.Sqlite;

class Program
{
    static void Main()
    {
        string connectionString = "Data Source=bank.db";

        using var connection = new SqliteConnection(connectionString);
        connection.Open();

        var create = connection.CreateCommand();
```

```
create.CommandText = @"
    CREATE TABLE IF NOT EXISTS Accounts (
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        Name TEXT,
        Balance REAL
    );";
create.ExecuteNonQuery();

var countCmd = connection.CreateCommand();
countCmd.CommandText = "SELECT COUNT(*) FROM Accounts";
long count = (long)countCmd.ExecuteScalar();
if (count == 0)
{
    var seed = connection.CreateCommand();
    seed.CommandText = "INSERT INTO Accounts (Name, Balance) VALUES ('A', 5000), ('B', 2000)";
    seed.ExecuteNonQuery();
}

using var transaction = connection.BeginTransaction();
try
{
    Console.WriteLine("Transferring 1000 from A → B...");

    var debit = connection.CreateCommand();
    debit.CommandText = "UPDATE Accounts SET Balance = Balance - $amt WHERE Name = 'A'";
    debit.Parameters.AddWithValue("$amt", 1000);
    debit.Transaction = transaction;
    debit.ExecuteNonQuery();

    var credit = connection.CreateCommand();
    credit.CommandText = "UPDATE Accounts SET Balance = Balance + $amt WHERE Name = 'B'";
    credit.Parameters.AddWithValue("$amt", 1000);
    credit.Transaction = transaction;
    credit.ExecuteNonQuery();

    transaction.Commit();
    Console.WriteLine("Transfer successful.");
}
catch (Exception ex)
{
    Console.WriteLine("Error: " + ex.Message);
    transaction.Rollback();
    Console.WriteLine("Transaction rolled back.");
}
var select = connection.CreateCommand();
select.CommandText = "SELECT Name, Balance FROM Accounts";
using var reader = select.ExecuteReader();
Console.WriteLine("\nFinal Balances:");
while (reader.Read())
    Console.WriteLine($"{reader.GetString(0)} → {reader.GetDouble(1)}");
```

```
    }
}
```

Question 7 : SQL Injection Attack Simulation

```
using System;
using Microsoft.Data.Sqlite;

class Program
{
    static void Main()
    {
        string connectionString = "Data Source=security.db";

        using var connection = new SqliteConnection(connectionString);
        connection.Open();
        var create = connection.CreateCommand();
        create.CommandText = @"
            CREATE TABLE IF NOT EXISTS Users (
                Id INTEGER PRIMARY KEY AUTOINCREMENT,
                Username TEXT,
                Password TEXT
            );";
        create.ExecuteNonQuery();

        var insert = connection.CreateCommand();
        insert.CommandText = "INSERT INTO Users (Username, Password) VALUES ('admin','secret')";
        insert.ExecuteNonQuery();

        Console.WriteLine("Try login with: username=admin, password=secret");
        Console.WriteLine("Or try injection: password=' OR 1=1 --");

        Console.WriteLine("\n--- Vulnerable Login ---");
        VulnerableLogin(connection, "admin", "' OR 1=1 --");

        Console.WriteLine("\n--- Secure Login ---");
        SecureLogin(connection, "admin", "' OR 1=1 --");
    }
    static void VulnerableLogin(SqliteConnection conn, string user, string pass)
    {
        var cmd = conn.CreateCommand();
        cmd.CommandText = $"SELECT COUNT(*) FROM Users WHERE Username='{user}' AND
Password='{pass}'";
        long count = (long)cmd.ExecuteScalar();
        Console.WriteLine(count > 0 ? "Login SUCCESS (vulnerable)" : "Login FAILED");
    }
    static void SecureLogin(SqliteConnection conn, string user, string pass)
    {
        var cmd = conn.CreateCommand();
        cmd.CommandText = "SELECT COUNT(*) FROM Users WHERE Username=$u AND Password=$p";
        cmd.Parameters.AddWithValue("$u", user);
```

```
        cmd.Parameters.AddWithValue("$p", pass);
        long count = (long)cmd.ExecuteScalar();
        Console.WriteLine(count > 0 ? "Login SUCCESS (secure)" : "Login FAILED");
    }
}
```

Vulnerable code → unsafe string concatenation.

Secure code → parameterized queries prevent injection.

Question 8 : Thread-Safe Singleton Database Manager

```
using System;
using System.Threading;
using Microsoft.Data.Sqlite;

class DatabaseManager
{
    private static readonly Lazy<DatabaseManager> _instance =
        new Lazy<DatabaseManager>(() => new DatabaseManager());

    public static DatabaseManager Instance => _instance.Value;

    public SqliteConnection Connection { get; }

    private DatabaseManager()
    {
        Console.WriteLine($"DatabaseManager instance created by Thread
{Thread.CurrentThread.ManagedThreadId}");
        Connection = new SqliteConnection("Data Source=singleton.db");
        Connection.Open();
    }
}

class Program
{
    static void Main()
    {
        Thread t1 = new Thread(() =>
        {
            var db = DatabaseManager.Instance;
            Console.WriteLine($"Thread {Thread.CurrentThread.ManagedThreadId} using instance
{db.GetHashCode()}");
        });

        Thread t2 = new Thread(() =>
        {
```

```
            var db = DatabaseManager.Instance;
            Console.WriteLine($"Thread {Thread.CurrentThread.ManagedThreadId} using instance
{db.GetHashCode()}");
        });

        Thread t3 = new Thread(() =>
        {
            var db = DatabaseManager.Instance;
            Console.WriteLine($"Thread {Thread.CurrentThread.ManagedThreadId} using instance
{db.GetHashCode()}");
        });

        t1.Start();
        t2.Start();
        t3.Start();

        t1.Join();
        t2.Join();
        t3.Join();
    }
}
```

Question 9 : Extend Factory with Open-Closed Principle

```
using System;
using System.Collections.Generic;

interface IShape
{
    void Draw();
}

class Circle : IShape
{
    public void Draw() => Console.WriteLine("Drawing Circle");
}

class Square : IShape
{
    public void Draw() => Console.WriteLine("Drawing Square");
}

class ShapeFactory
{
    private readonly Dictionary<string, Func<IShape>> _registry =
        new Dictionary<string, Func<IShape>>(StringComparer.OrdinalIgnoreCase);

    public ShapeFactory()
    {
```

```
        Register("Circle", () => new Circle());
        Register("Square", () => new Square());
    }

    public void Register(string key, Func<IShape> creator)
    {
        _registry[key] = creator;
    }

    public IShape Create(string key)
    {
        if (_registry.TryGetValue(key, out var creator))
            return creator();
        throw new ArgumentException($"Shape '{key}' not registered.");
    }
}

class Program
{
    static void Main()
    {
        var factory = new ShapeFactory();

        factory.Register("Triangle", () => new Triangle());

        IShape s1 = factory.Create("Circle");
        IShape s2 = factory.Create("Square");
        IShape s3 = factory.Create("Triangle");

        s1.Draw();
        s2.Draw();
        s3.Draw();
    }
}

class Triangle : IShape
{
    public void Draw() => Console.WriteLine("Drawing Triangle");
}
```

Question 10 : **Observer-Based Stock Market (Async Notification)**

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;

interface IObserver
{
```

```
    Task UpdateAsync(int price);
}

class EmailSubscriber : IObserver
{
    public async Task UpdateAsync(int price)
    {
        await Task.Run(() =>
        {
            Console.WriteLine($"durgesh12@gmail.com Stock price updated to {price}");
        });
    }
}

class SmsSubscriber : IObserver
{
    public async Task UpdateAsync(int price)
    {
        await Task.Run(() =>
        {
            Console.WriteLine($"9876543210 Stock price updated to {price}");
        });
    }
}

class Stock
{
    private readonly List<IObserver> observers = new();

    public void Subscribe(IObserver observer) => observers.Add(observer);

    public async Task SetPriceAsync(int price)
    {
        Console.WriteLine($"Stock price set to {price}");

        var tasks = new List<Task>();
        foreach (var observer in observers)
        {
            tasks.Add(NotifyObserver(observer, price));
        }

        try
        {
            await Task.WhenAll(tasks);
        catch (Exception ex)
        {
            Console.WriteLine("Error notifying observers: " + ex.Message);
            Console.WriteLine("Trace: " + ex.StackTrace);
        }
    }
```

```
    private async Task NotifyObserver(IObserver observer, int price)
    {
        try
        {
            await observer.UpdateAsync(price);
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Observer failed: {ex.Message}");
        }
    }
}

class Program
{
    static async Task Main()
    {
        Stock stock = new Stock();
        stock.Subscribe(new EmailSubscriber());
        stock.Subscribe(new SmsSubscriber());

        await stock.SetPriceAsync(120);
        await stock.SetPriceAsync(150);

        Console.WriteLine("Main thread continues without blocking.");
    }
}
```